



# Datenbank für alle Fälle

## Mehr als SQL: Die richtige Datenbank für Ihr Projekt

Ihre Software soll Daten speichern und schnell wiederfinden und am besten sollen auch mehrere Nutzer gleichzeitig darauf zugreifen? Dann muss eine Datenbank her – nur welche? Ein Streifzug durch das Angebot, der bekannte Klassiker sortiert, Spezialisten vorstellt und die Strategien der Neulinge einordnet.

Von Jan Mahn

**E**in neues Softwareprojekt steht an. Voller Vorfreude stürzen Sie sich in die Planung: Die Programmiersprache steht

schnell fest, auch auf ein Framework haben Sie sich mit Ihren Kollegen rasch geeinigt. Wie bei jedem neuen Projekt haben Sie sich geschworen, diesmal die Fehler vergangener Entwicklungen nicht zu machen und jeden Schritt gründlich zu überlegen. Schnell ist das Fundament angelegt, ein Git-Repository erstellt und Sie erreichen den Punkt, an dem das junge Softwaregerüst die ersten Daten speichern soll. „Binde doch schnell eine SQLite-Datetei ein und entwickle erstmal damit, da hängen wir dann später eine MySQL- oder PostgreSQL-Datenbank dran“, schlägt ein Kollege vor. Halt, hatten Sie sich nicht darauf geeinigt, diesmal jeden Schritt sorgfältig zu durchdenken? Sind MySQL oder Post-

greSQL wirklich die beste Wahl für das Projekt, nur weil das Gehirn Ihres Kollegen bei der Abfrage

```
SELECT * FROM databases
ORDER by popularity DESC LIMIT 2
```

ausgerechnet diese beiden Treffer ausgespuckt hat? Oder ist es an der Zeit, mal über den Tellerrand zu blicken? Der Markt an Datenbankmanagementsystemen

(DBMS) ist groß – viele sind quelloffen, einige kann man bequem online testen, bevor man sie selbst installiert. Einige sind eher Generalisten, andere hochspezialisiert auf eine Aufgabe. Und gleichzeitig ist der Markt ständig in Bewegung, Start-ups



kommen, Firmen werden verkauft, Open-Source-Projekte wechseln die Lizenz und Forks spalten sich ab. Als Entwickler ist es schwer, da den Überblick zu behalten. Der folgende Wegweiser ist keinesfalls vollständig – bei Dutzenden aktiv entwickelten DBMS ist das nicht möglich. Er soll vielmehr eine Entscheidungshilfe darstellen und Trends aufzeigen.

## SQL-Klassiker

Rankings zur Popularität von Software, die im Internet kursieren, sind immer mit etwas Vorsicht zu genießen. Die Website DB-Engines (db-engines.com) stellt solche Bestenlisten bereit und berechnet die Popularität von Datenbanken anhand von Suchergebnissen, Google Trends, Diskussionen auf Stack Overflow, Erwähnungen in Stellenanzeigen und sozialen Netzwerken. Die Frage-Antwort-Plattform Stack Overflow führt zusätzlich jährlich eine große Entwicklerumfrage durch und erfasst dabei auch Datenbanken (Survey von 2022 zu finden über [ct.de/y1n6](https://ct.de/y1n6)). So problematisch jede dieser Methoden auch sein mag, in einem sind sich beide seit Jahren einig: Klassische relationale Datenbanken führen die Rankings immer an.

Daran ist auch nichts verkehrt, relationale Datenbanken sind oft die beste oder zumindest eine sehr gute Wahl für viele Arten von Daten. Wenn man zum Beispiel eine kleine Anwendung plant, in der Mitarbeiter Konferenzräume buchen sollen, muss man Objekte wie Benutzer, Gruppe, Raum, Etage und Raumbuchung verwalten. Zwischen diesen gibt es in der echten Welt Verbindungen und in einer relationalen Datenbank passende Relationen. Ein Benutzer kann mehreren Gruppen zugewiesen sein, ein Raum nur einer Etage. Eine typische Frage an die Datenbank: Welche Raumbuchungen stehen auf Etage 2 heute an? Mit JOIN-Konstruktionen ist sie schnell beantwortet.

Welche relationale Datenbank darf es jetzt sein? Auf der Vergleichswebsite DB-Engines führt die **Oracle-Datenbank** im September 2023 und liefert sich seit über 10 Jahren ein enges Rennen mit **MySQL** und **Microsofts SQL Server**. MySQL gewann auch in der Umfrage von Stack Overflow aus 2022. Alle drei bieten relationale Standardkost mit leichten Unterschieden in der SQL-Syntax, SQL-Dialekte genannt. Solche Dialekte sind Abweichungen von der reinen Lehre, die im internationalen Standard ISO/IEC 9075-3:2023 niedergeschrieben sind. In Microsofts Mundart

## ct kompakt

- Relationale Datenbanken sind die erste Wahl für viele Anwendungen und erledigen ihre Aufgaben solide.
- Oft ist es für Entwicklung und Performance sinnvoll, spezielle Aufgaben auf andere Systeme wie Zeitreihendatenbanken oder Schlüssel-Werte-Speicher auszulagern.
- Selbstbetriebene Datenbanken sind nur noch ein Modell unter mehreren. Die Software kann auch im Hosting-Paket gemietet werden.

namens T-SQL lautet obenstehende Abfrage beispielsweise:

```
SELECT TOP 2 * FROM databases
ORDER by popularity DESC
```

## Geschickt abstrahiert

Die Zeiten, in denen solche Feinheiten Entwickler und insbesondere Umsteiger in den Frust getrieben haben, sind bei vielen Projekten längst vorbei; die meisten Anwendungen enthalten keine einzige Zeile handgestricktes SQL mehr. Stattdessen greifen Entwickler direkt zu einem ORM (Object-Relational Mapping), einer Abstraktionsschicht, die zwischen Objekten im Programmcode und der Datenbank vermittelt. Mit der Definition von Datenbanktabellen und Relationen haben sie es dann nur noch an einer Stelle zu tun, in sogenannten Migrationen. Darin erklärt man seiner Software, wie die Tabellen aufgebaut sind und welche Objekte sie im Programm repräsentieren. Danach kann man in der Syntax der jeweiligen Programmiersprache oder des Frameworks auf die Datenbank zugreifen. Deutlich wird das zum Beispiel anhand des ORMs Eloquent aus der PHP-Welt. Sobald man einmal festgelegt hat, dass das Objekt `MyDatabase` zur Tabelle `databases` gehört, würde man obenstehende Abfrage so formulieren:

```
$dbs = MyDatabase::orderBy(
    'popularity', 'DESC')->take(2)->get();
```

In der Konfiguration des Projekts hinterlegt man die Adresse, die Zugangsdaten und den Typ der Datenbank. Dass ein Microsoft-SQL-Server gern `TOP 2` hätte, MySQL

mit `LIMIT 2` angewiesen werden will, haben die Entwickler des ORMs berücksichtigt und übersetzen im Hintergrund. Es wäre jedoch naiv anzunehmen, dass das Grundwissen über relationale Datenbanken und die sogenannte Normalisierung nun obsolet ist, weil das ORM die ganze Magie erledigt. Wer so an die Entwicklung geht, baut garantiert langsame Anwendungen und in sich kaputte Datenstrukturen.

Für Entwickler eröffnen ORMs dennoch viele Freiheiten, wenn sie sich einmal dafür entschieden haben, mit relationalen Datenbanken zu arbeiten. In gewissem Rahmen können sie sich auch während der Entwicklung noch für eine andere relationale Alternative entscheiden, ohne den Code anfassen zu müssen – oder ihr Produkt gar so gestalten, dass der Kunde später seine eigene Lieblingsdatenbank nutzen kann.

Problematisch wird der Wechsel zwischen ähnlichen Datenbanken immer dann, wenn man seinem ORM innerhalb der Migration mitteilt, dass man für eine Tabellenspalte gern einen Datentyp hätte, den es nur in einem ganz bestimmten DBMS gibt. Mit jeder Sonderlocke, die ein DBMS anbietet, erhöht es den Vendor-Lock-in-Effekt.

Eine vor allem bei Webentwicklern lange populäre Strategie ist seit dem Siegeszug von Containern etwas auf dem Rückzug: Wer schnell einen ersten Prototypen bauen wollte, ohne sich mit der Administration einer Entwicklungsinstanz der Datenbank herumzuzergern, griff gern zu **SQLite**, einer Bibliothek, die ein „richtiges“ DBMS imitiert, aber keinen Serverdienst hochfährt. Stattdessen liegen die Daten in einer schlichten lokal gespeicherten Datei. Weil auch viele ORMs mit SQLite zusammenarbeiten, können Ungeduldige sofort losprogrammieren. Seit eine lokale MySQL- oder MariaDB-Instanz nur ein `docker run` oder `podman run` entfernt sind, geht es mit derjenigen Datenbank, die später auf dem produktiven Server laufen soll, genauso schnell und komfortabel.

Ist die Wahl der relationalen Datenbank eine reine Geschmacksfrage, wo doch ORMs alles abstrahieren? Mitnichten. Abseits privater Projekte haben diejenigen ein Wort mitzureden, die die Datenbanken am Ende betreiben und diejenigen, die sie bezahlen müssen. Die vielen zahlenden Oracle-Nutzer, auf die die Rankings hinweisen, dürften in der Regel größere Unternehmen sein, die traditionell Oracle-Produkte einsetzen. Start-ups und Betreuer von kleineren Projekten dürften nur

wenig Lust verspüren, sich in den Lizenz-dschungel dieses Herstellers einzuarbeiten. Auch die Zielgruppe für Microsoft SQL Server ist klar: Organisationen, die schon tief im Microsoft-Kosmos stecken und mit dem Lizenzsystem vertraut sind. Aber unter CEO Satya Nadella heißt Microsoft eben schon lange nicht mehr automatisch Windows (Server). Der SQL Server läuft auch unter Linux, ist für RHEL, SUSE Enterprise und Ubuntu ebenfalls getestet und für produktive Anwendungen freigegeben. Außerdem gibt es einen Linux-Docker-Container. Diese Software bildet auch die technische Grundlage für ein anderes Angebot aus dem Hause Microsoft: „Azure SQL Database“, eine Plattform-as-a-Service. Kunden bekommen eine komplett vom Hersteller betreute Umgebung, müssen sich also nicht um das Administrationsgeschäft kümmern. Mehr zu Varianten von Datenbank-Hosting lesen Sie im Kasten unten.

### Meine oder Marias?

Unter den kostenlosen Datenbanken landet man schnell bei MySQL oder MariaDB, die eng verwandt sind. MySQL war zuerst da, wurde 2008 von Sun Microsystems übernommen und dann 2010 von Oracle geschluckt. Schon nach dem Kauf durch

Sun legte der ursprüngliche MySQL-Erfinder Michael Widenius den Fork namens **MariaDB** unter der Lizenz GPL an und plante zunächst, eine vollständig kompatible Alternative zu pflegen.

Unter Oracle als neuem Besitzer kippte die Stimmung in der Open-Source-Szene dann zunehmend und die Kritik wurde lauter, dass Oracle neue Funktionen ausschließlich in die Enterprise-Version von MySQL einbaue. Anfangs war ein Umstieg von MySQL zu MariaDB auch ein politisches Statement gegen das verhasste Oracle, dann folgten immer mehr Linux-Distributionen und machten MariaDB zum Standard. In den aktuellen Versionen (MariaDB 10.9 und MySQL 8.0) haben sich die beiden DBMS in Details auseinandergeliebt. Viele Unterschiede kann ein ORM ausgleichen, beim Umziehen von Daten aus einem System in ein anderes kann es aber knirschen – die Entscheidung für ein System muss also am besten fallen, bevor man Daten produziert. Etwas unterschiedlich verhalten sich die beiden zum Beispiel beim Umgang mit UUIDs, wie bereits ausführlich vorgestellt [1], oder bei JSON-Daten.

### Rückabwicklung

Doch beide DBMS eint auch sehr viel. Auf der einen Seite nutzen sie im Hintergrund

beide in der Voreinstellung das Datenbanksystem InnoDB, das die Daten auf die Festplatte schreibt und davon ausliest. Auch wenn man als Entwickler nicht direkt mit InnoDB zu tun hat, ist diese Information entscheidend: Denn InnoDB beherrscht sogenannte Transaktionen, auf die man in vielen Anwendungen nicht verzichten mag. Was es damit auf sich hat, lesen Sie im Kasten „Darum Transaktionen“ auf Seite 100.

In früheren Zeiten fiel unweigerlich ein weiterer bekannter Name, wenn es um Datenbanken mit Transaktionen ging: **PostgreSQL** unterstützte das Transaktionsverfahren schon, als bei MySQL noch das Backend MyISAM zum Einsatz kam, das die Funktion nicht hatte. Daher hält sich bis heute hartnäckig die Behauptung, nur PostgreSQL sei transaktionsfähig. Dem ist nicht so, aber PostgreSQL hat andere Stärken, die es gegenüber der Konkurrenz attraktiv machen. Eine ist der Datentyp JSON, dank dem man strukturierte Daten im JSON-Format ablegen kann – und zwar nicht als reine Zeichenkette, sondern auch durchsuch- und manipulierbar. Ohne dieses Extra müsste man mühsam mit regulären Ausdrücken in JSON-Zeichenketten suchen, die man in die Datenbank gelegt hat. Ganz allein ist PostgreSQL aber auch mit dieser Funktion nicht mehr: MySQL hat nachgezogen und bietet ebenfalls einen JSON-Datentyp. MariaDB hat den Datentyp JSON neuerdings auch im Programm, unter der Haube ist es aber schlicht ein Alias für den Typ LONGTEXT.

In PostgreSQL verbergen sich aber noch andere nützliche Funktionen, mit denen die Datenbank heraussticht. Mit LISTEN und NOTIFY kann man den Spieß zum Beispiel umdrehen: Anstatt eine Software zu bauen, die alle paar Sekunden die Datenbank nervt, ob es etwas Neues gibt, richtet man Benachrichtigungen ein und PostgreSQL meldet sich, sobald etwas passiert. PostgreSQL eröffnet damit das weite Feld der Event-Driven-Architecture, die sich großer Beliebtheit erfreut, wenn man Software auf verteilten Systemen betreiben will (oder soll).

Für solche verteilten Umgebungen gibt es noch eine weitere DBMS-Alternative und einen guten Grund, seine Anwendung für Kompatibilität mit PostgreSQL zu optimieren. Die klassischen Datenbankmanagementsysteme stammen alle aus einer Zeit, in der es nicht üblich war, die Arbeit der Datenbankserver über meh-

## Hostingmodelle für Datenbanken

Das Vermieten von Datenbanken ist zu einem lukrativen Geschäft geworden; immer mehr Dienstleister bewerben sich als Alternative zum eigenen Server. Auf der einen Seite bieten viele Hersteller, die hinter einer Datenbank stehen, gehostete Datenbanken an, andererseits sind aber auch verschiedene Cloudanbieter früh ins Geschäft eingestiegen – nicht immer fair, wie die Entwickler der quell-offenen Software beklagen.

Hosting von Datenbanken gibt es in verschiedenen Spielarten, nicht jeder Anbieter hat alle im Programm. Am meisten Arbeit, aber auch am meisten Kontrolle hat man mit einer verwalteten Instanz auf selbst gewählter Cloudinfrastruktur. Dieses Modell ist sehr ähnlich zu einer selbstgehosteten Enterprise-Variante. Als Kunde mietet man selbst Rechenleistung und Speicherplatz und der Anbieter installiert und aktualisiert die Datenbankumgebung vollautomatisch. Als Adminis-

trator behält man die Kontrolle über das darunterliegende Betriebssystem.

Etwas weniger Kontrolle gewähren voll verwaltete Instanzen. Die virtuelle Maschine, auf der sie läuft, fährt der Anbieter bei Amazon, Google, Microsoft & Co. hoch und startet darin den Datenbankserver. Als Administrator bekommt man nur Zugriff auf die Kontrolloberfläche der Datenbank, nicht aber auf den Server darunter. Um Updates kümmert sich eine Automatik des Anbieters.

Gar keine Kontrolle, dafür niedrige Kosten oder gar ein kostenloses Paket: Dies charakterisiert die Serverless-Variante. Da hat man gar keinen eigenen Server, sondern teilt sich anonym große Instanzen. Man kann und muss sich nicht darum kümmern, wie viel CPU und RAM irgendwelche Maschinen verbrauchen. Einfach in einer Weboberfläche Datenbank und Benutzer anlegen und mit den Zugangsdaten losprogrammieren.

reere Maschinen oder gar (Cloud-)Rechenzentren zu verteilen. Master und Slaves waren lange das Mittel der Wahl, zwischen denen im Fehlerfall umgeschaltet wurde. Daher sind sämtliche Funktionen, um MariaDB, MySQL oder PostgreSQL redundant und verteilt laufen zu lassen, nachträglich über parallel entwickelte Projekte angebaut worden.

Ganz anders sieht das bei der modernen, ebenfalls quelloffenen Datenbank **CockroachDB** aus, die von Haus aus auf Betrieb in Containern optimiert ist, sich hervorragend in eine Docker- oder Kubernetes-Umgebung einfügt und die Daten redundant vorhält. Fällt mal ein Server aus, übernehmen die anderen nahtlos.

CockroachDB ist nicht Open Source im strengen Sinne, weil ihre Business Source License (BSL) ausschließt, dass man damit einen kommerziellen Datenbankhoster als Konkurrenz zum Angebot von Cockroach-Labs aufzieht. Ansonsten darf man mit der kostenfreien Variante aber alles anstellen, sie also insbesondere im Unternehmen einsetzen, an Kunden ausliefern und auch zusammen mit der eigenen Anwendung für Kunden betreiben.

Diese Semi-Open-Source-Lizenzen werden Ihnen auf unserem Streifzug noch an anderer Stelle begegnen. Open-Source-Puristen lehnen solche Einschränkungen strikt ab, die Entwickler verteidigen sich und kritisieren, dass große Anbieter wie Amazon ihre Datenbank fürs Hosting einsetzen, damit viel Geld verdienen, aber sich zu wenig an der Weiterentwicklung beteiligen. Amazons AWS stellt zum Beispiel unter dem Produktnamen Amazon RDS unter anderem PostgreSQL, **MariaDB** und MySQL bereit.

Längst ist CockroachDB kein Geheimtipp mehr, auch weil die Entwickler es Umsteigern leicht gemacht haben: Anstatt das Rad neu zu erfinden, nutzen sie für die Kommunikation mit der Datenbank dasselbe Protokoll wie PostgreSQL, sprechen denselben Dialekt. Sofern eine Programmiersprache für PostgreSQL-Kommunikation ausgelegt ist, können Sie auch CockroachDB ansprechen. Wenn das ORM noch keine offizielle Unterstützung für diese Datenbank hat, trägt man PostgreSQL in die Konfiguration ein – oft kommt man damit sehr weit und kann versuchen, die eigene Anwendung für einen Betrieb in verteilten Systemen flottzumachen. Manchmal klemmt es auch hier, denn zu 100 Prozent haben die CockroachDB-Entwickler den Funktionsumfang des

The screenshot shows the Amazon RDS website. At the top, there's a navigation bar with links like 'Amazon RDS', 'Übersicht', 'Funktionen', 'DB Engines', 'Preise', 'Ressourcen', 'Häufig gestellte Fragen', 'Kunden', and 'Partner'. The main heading is 'Amazon Relational Database Service'. Below it, a subheading says 'Richten Sie mit nur wenigen Klicks eine relationale Datenbank in der Cloud ein und betreiben und skalieren Sie sie.' To the right, a box highlights 'Kostenfreier Einstieg mit dem kostenlosen AWS-Kontingent'. Below the heading, there are two buttons: 'Erste Schritte mit Amazon RDS' and 'Mit einem Amazon-RDS-Experten in Kontakt treten'. The main content area is divided into four columns, each with a benefit: 1. 'Beseitigen Sie ineffiziente und zeitaufwändige Datenbank-Verwaltungsaufgaben, ohne dass eine Infrastruktur bereitgestellt oder Software gewartet werden muss.' 2. 'Stellen Sie relationale Datenbank-Engines Ihrer Wahl in der Cloud oder On-Premises bereit und skalieren Sie sie.' 3. 'Erzielen Sie eine hohe Verfügbarkeit dank der Verwendung von Amazon RDS in mehreren AZs.' 4. 'Profitieren Sie von mehr als einem Jahrzehnt Betriebserfahrung, bewährten Sicherheitsmethoden und Innovationen im Bereich der Cloud-Datenbanken.' Below this, there's a section titled 'Funktionsweise' with a brief description of Amazon RDS services.

**Amazon handelt nicht nur mit Büchern und Haushaltswaren. Im Cloudangebot AWS vermietet das Unternehmen auch gehostete Open-Source-Datenbanken wie MySQL und PostgreSQL.**

Vorbilds noch nicht nachgebaut. Wenn Sie neugierig sind und ohne Administrationsaufwand loslegen wollen: 10 Gigabyte Speicherplatz und 50 Millionen Anfragen im Monat gibt es in der Serverless-Version kostenlos in der Cloud (zu finden über [ct.de/y1n6](#)).

## Nicht nur SQL

Es ist Zeit, die breite Hauptstraße der relationalen Datenbanken zu verlassen und in eine Nebenstraße abzubiegen, die sich NoSQL nennt. Das soll heute aber keinesfalls „no SQL“ bedeuten, sondern vielmehr „not only SQL“ – für viele Projekte ein sinnvoller Ansatz: Anstatt zu versuchen, relationale Datenbank zu ersetzen, kann es sinnvoll sein, das Kerngeschäft (wie eine Nutzer- und Gruppenverwaltung) in einer solchen Datenbank zu lagern und Spezialaufgaben an eine darauf ausgelegte Datenbank zu delegieren, die den Job schneller und eleganter erledigt. Fast alles, was die im Folgenden vorgestellten Systeme bieten, kann man auch einer MariaDB beibringen – aber nicht immer schön.

Die erste Assoziation zum Begriff NoSQL ist für viele **MongoDB**, ein Exemplar aus der Art der schemalosen dokumentengestützten Datenbanken. Das Konzept erfreut sich besonders im Bereich der Webentwicklung großer Beliebtheit; in einer Umfrage von Stack Overflow aus dem Jahr 2022 landete MongoDB auf Platz

4. Die Grundidee: Anstatt für ein Objekt, das man speichern möchte, zuerst per SQL eine Tabelle mit Spalten und Datentypen anzulegen, wirft man gleiche oder ähnliche Datenobjekte in eine sogenannte Collection. Die Datenbank beklagt sich aber nicht, wenn ein neues Objekt plötzlich ein Attribut mehr oder weniger hat. Man muss sich nicht wie in der SQL-Welt darum kümmern, eine Spalte zu entfernen, NULL- oder Default-Werte zu definieren. Vorstellen kann man sich eine Collection wie einen Ordner auf der Festplatte, der JSON-Dateien enthält.

Bedient wird MongoDB über Softwarebibliotheken, die für viele Programmiersprachen bereitstehen und die eine Syntax mit Methoden wie `.find()` und `.insert_one()` anbieten, für die man bei einer SQL-Datenbank ein ORM einsetzen muss. Gleichwohl gibt es ORMs, die auch mit MongoDB sprechen – wie das oben schon als Beispiel benutzte Eloquent. Wenn Sie sich auf das Abenteuer einlassen und Ihre nächste Anwendung mal mit einer dokumentengestützten Datenbank statt mit einer relationalen bauen wollen, stellen Sie sich darauf ein, dass Sie beim Anlegen von Relationen kräftig umlernen müssen – Beziehungen sind nicht unmöglich, funktionieren mit „Embedded Documents“ und „Document References“ aber anders als in MariaDB & Co.

MongoDB Inc. war 2018 eines der ersten Unternehmen, das die Lizenz seiner

## Darum Transaktionen

Transaktionen helfen dabei, die sogenannten ACID-Kriterien zu erfüllen, die beschreiben, dass Änderungen vollständig und dauerhaft in der Datenbank liegen. Die Kurzform an einem Beispiel: Eine Banksoftware soll eine Überweisung von einem Konto auf ein anderes schreiben. Zuerst vermindert sie den Kontostand des einen Kontos um 50 Euro und soll ihn später auf dem anderen Konto erhöhen. Jetzt tritt ein Fehler auf – der Server stürzt zum Beispiel mittendrin ab. Oder die Anwendung merkt plötzlich, dass der Dispokredit überzogen ist und sie das Geld nicht übertragen darf. In solchen Fällen muss der ganze Vorgang rückabgewickelt werden, damit keine fehlerhaften Relikte in der Datenbank verbleiben. Sonst hätte die Anwendung Geld verschwinden lassen. Der Trick: Man hüllt die einzelnen Schritte in eine TRANSACTION, und nur wenn es keine Probleme gab, führt man am Ende den Befehl COMMIT aus. Es wird also immer nur alles oder nichts geändert. Bei Problemen kommt es zum ROLLBACK.

Software änderte und von Open Source auf eine quelloffene Lizenz mit Einschränkungen umstieg – dementsprechend heftig empörte sich damals die Community. Die neue Lizenz heißt „Server Side Public License“ und verbietet kostenpflichtige Hosting-Dienste. Solche möchte das Unternehmen gern selbst anbieten.

### Schlüssel zum Wert

Eine andere schemafreie Alternative zu den relationalen sind die Key-Value-Datenbanken, die im Kern nur eine Sache können: unter einem Schlüssel Werte ablegen und wieder auslesen. Mit einer einfachen Datenbanktabelle und den Spalten `key` und `value` könnte man diese Fähigkeit auch einer SQL-Datenbank abringen. Ein Cachesystem ist ein typischer Anwendungsfall für eine Key-Value-Datenbank, ebenso ein zentraler Speicherort für Konfigurationen. Der bekannteste Vertreter seiner Art heißt **Redis** und kann weit mehr als nur lesen und schreiben. Redis ist veröffentlicht unter der „3-Clause BSD License“ und damit Open Source, gleichermaßen gibt es Erweiterungen mit Einschrän-

kungen und auch eine kommerzielle Enterprise-Version.

Weil die Datenstruktur von Redis schemalos ist, müssen Sie nicht spezifizieren, wenn Sie unter dem Schlüssel `website:cache:welcome` einen String und unter `website:counter:welcome` einen Integer ablegen wollen. Der Schlüssel wird angelegt, sobald Sie die ersten Daten schreiben. Zum Experimentieren empfiehlt sich eine interaktive Kommandozeilensitzung mit einer einfachen Befehlsyntax:

```
set website:cache:welcome 2
"Das ist ein Test"
```

Mit `get website:counter:welcome` bekommen Sie den Inhalt des Schlüssels zurück. Ebenso leicht ist es, mit dem Befehl `incr` zum Beispiel einen Zähler hochzuzählen:

```
set website:counter:welcome 0
incr website:counter:welcome
```

Das beeindruckt Sie als SQL-Nutzer noch nicht? Als Beispiel, was Redis noch so kann, soll der Eintrag des Cache nach 10 Sekunden automatisch gelöscht werden:

```
EXPIRE website:cache:welcome 10
```

Und das ist längst nicht alles. Die Dokumentation verrät, was es sonst noch zu entdecken gibt: zum Beispiel Listen, mit denen man sehr leicht Worker-Queues bauen kann. Wenn Sie Redis ganz ohne Installation ausprobieren wollen, testen Sie den Online-Spielplatz `try.redis.io` und setzen die Befehle im Browser ab. In Ihrem ersten Softwareprojekt schreiben Sie dann keine Befehle wie in der Kommandozeilensitzung. Stattdessen nutzen Sie eine Redis-Bibliothek, die es für jede gängige Programmiersprache gibt und erhalten so eine objektorientierte Syntax wie zum Beispiel diese in Python:

```
r.set('schluessel', 'inhalt')
```

### Zeitreihen

Ein anderer Sonderfall sind Zeitreihen, die prinzipiell auch mit einer relationalen Datenbank denkbar sind. Angenommen, Sie verarbeiten Sensordaten von Dutzenden Wetterstationen, die alle nacheinander in Ihrem System ankommen. Solche Werte wollen Sie mit Datum und Uhrzeit wegschreiben. Später wollen Sie mit die-

sen Daten etwas Nützliches anstellen – etwa Tageshöchst- und Tiefstwerte oder Stundenmittel berechnen. Mit einer relationalen Datenbank sowie ausgefeilten SQL-Abfragen oder einer Menge Code geht das alles problemlos. Aber mit der Zeit werden die Datenmengen größer und die Performance bricht ein. Sie merken, dass Sie Gigabyte voller Datenmüll in Form von minütlichen Werten lagern, wo Sie doch eigentlich nur die Durchschnittswerte der vergangenen Tage brauchen. Skripte basteln, die Werte der Vergangenheit aggregieren und die Rohdaten löschen? Das geht eleganter, nämlich mit Zeitreihendatenbanken (Time Series Databases).

Stellvertretend für diese Gattung ist **InfluxDB**, erschienen als Open-Source-Variante unter MIT-Lizenz. Wie in der Branche üblich, gibt es eine Enterprise-Edition und gehostete Angebote, betrieben von der Firma InfluxData. Um InfluxDB schnell einem großen Entwicklerpublikum bereitzustellen, haben die Entwickler darauf verzichtet, ein eigenes Protokoll zur Kommunikation mit der Datenbank auf TCP/IP zu stricken. Stattdessen haben sie sich nur ein sogenanntes Line Protocol – also eine formatierte Zeile Text – ausgedacht, die Clients per HTTP am API der Datenbank abliefern. Eine InfluxDB-Klasse für eine Programmiersprache zu programmieren war daher leicht, weil diese nur einen HTTP-Client abstrahiert. Deshalb gibt es die InfluxDB-Klasse nicht nur für gängige Programmiersprachen, sondern sogar für den Arduino.

Die Grundidee von InfluxDB in der Version 2: Daten eines Typs (zum Beispiel alle Messwerte der Wetterstationen) kommen in ein Bucket. Ein Datenpunkt, den zum Beispiel der Sensor abliefern, heißt Measurement und enthält Tags (Schlüssel-Werte-Paare mit Metadaten) und Fields (ebenfalls Schlüssel-Werte-Paare mit den Messungen). Mit Fields kann man später rechnen, auf Tags kann man effizient filtern. Die Magie von InfluxDB entfaltet sich an mehreren Stellen. Auf der einen Seite ist da die eigene Abfragesprache Flux, die zum Beispiel darauf ausgelegt ist, mit Zeiträumen zu operieren. Anstatt viel zu rechnen, reicht in dieser Syntax zum Beispiel `range(start: -2d)`, um alle Werte der letzten zwei Tage zu filtern.

Die andere Stärke sind die Funktionen zum automatischen Downsampling. Dabei werden Daten zusammengefasst und alte Datenpunkte automatisch ent-

sorgt. Mit diesen Fähigkeiten hat InfluxDB viele Freunde gefunden, die nicht nur Bibliotheken, sondern auch Anbindungen an andere Software gebaut haben. Daten aus InfluxDB kann man zum Beispiel bequem in Grafana visualisieren.

## Raus aus der JOIN-Hölle

Relationale Datenbanken sind stark darin, Verknüpfungen über Tabellen herzustellen, wie oben im Beispiel des Raumplans angerissen. Doch solche Relationen haben Grenzen, an die bereits stößt, wer seiner Datenbank Zusammenhänge über mehrere Ecken entlocken will. Der Betreiber eines Webshops zum Beispiel interessiert sich im Rahmen einer Marketingaktion dafür, wer Produkte einer Kategorie, die wiederum zu einer Überkategorie gehört, gekauft hat und welche anderen Bestellungen dieser Kunde in den letzten Wochen noch so erledigt hat. In SQL endet dieses Unterfangen in JOIN-Kaskaden und manche Probleme kann man nur in mehreren Abfragen lösen – wenn überhaupt. Wie heißen denn alle Freunde der Freunde der Freunde und nach wie vielen JOINS sollte man aufhören?

Auf solche Aufgaben haben sich Graphdatenbanken spezialisiert. Die speichern Objekte (Nodes) mit Attributen,

sowie ihre Beziehungen (Edges) zueinander. An diesem Verhältnis kann man wiederum Informationen anheften. Visualisiert man die Inhalte, ergibt sich ein Netz, die Beziehungen stellen Linien zwischen den Knoten dar. Ausgestattet sind solche Datenbanken mit einer eigenen Abfragesprache und mit Algorithmen, die die Pfade entlangwandern können.

Die vielleicht bekannteste Graphdatenbank heißt **Neo4J** und erscheint auch als Open-Source-Version. Im Cluster, also auf mehreren Servern, können Sie diese kostenlose Variante aber leider nicht betreiben. Eine etwas unbekanntere Alternative heißt **Dgraph**, dessen Open-Source-Version auch im Clusterbetrieb arbeitet. Allerdings müssen Sie die Dgraph Query Language (DQL) lernen, um Anfragen zu stellen. Offizielle und inoffizielle Bibliotheken für Programmiersprachen gibt es reichlich und im Notfall gilt, was auch für InfluxDB gilt: Die Kommunikation stützt sich auf HTTP und ist daher auch ohne extra Bibliothek möglich.

## Suchen in Texten

Dass Datenbank-Erfinder ihre Lizenzen ändern und nicht mehr ganz so open sind: Daran hat man sich schon gewöhnt. Dass daraus direkt ein Fork entsteht, und ein

erfolgreicher noch dazu, ist jedoch nicht alltäglich. So widerfuhr es dem Unternehmen Elastic, nachdem es 2021 seine Datenbank **Elasticsearch** auf die Elastic License v2 umgestellt hatte – als Reaktion auf Amazons „nicht-kollaboratives Verhalten und den Missbrauch unserer Marke“, wie Elastic schrieb. Der Fork **OpenSearch** entwickelt sich seitdem parallel unter der Apache License.

Zu OpenSearch oder Elasticsearch wollen Sie immer dann greifen, wenn Sie große Datenmengen, am liebsten mit viel Text, durchsuchbar lagern wollen. Eine Suchmaschine für eine Website oder die firmeninterne Datenzentrale ist ein gutes Anwendungsszenario für eine der beiden Datenbanken. Wenn Sie jemals mit `%LIKE%` in SQL an die Grenzen gestoßen sind, sind Elasticsearch und OpenSearch vielleicht der Heilsbringer. Normalerweise ersetzen sie andere Datenbanken aber nicht komplett, sondern dienen eher als zusätzliche Datenhalde. Sie werden aus verschiedenen Quellen mit zu speichernden Objekten beworfen, legen diese in ihrer Struktur ab und fischen sie auf Wunsch zügig wieder heraus.

## Fazit

Wer nur einen Hammer hat, sieht in jedem Problem einen Nagel – und so ist es auch mit relationalen Datenbanken. Fast alle Probleme kann man irgendwie in Tabellen, Spalten und Relationen verpacken und mit SQL lösen. Dabei lohnt sich bei vielen Aufgaben ein Blick auf spezialisierte Alternativen. Die meisten kann man spielend leicht online testen, ohne einen Dienst selbst hochzufahren.

Wenn man zum Beispiel die Worker-Queue und den Cache in eine Key-Value-Datenbank und die Messdaten in eine Zeitreihendatenbank ausgelagert hat, erledigt die relationale Datenbank nur noch das, was sie richtig gut kann: Objekte und ihre Relationen verwalten und auslesen. Wer sich dann von rohen SQL-Aufrufen in einem datenbankspezifischen Dialekt verabschiedet und ein ORM einsetzt, muss auch weniger Angst vor durchaus möglichen Lizenzänderungen haben und hält sich den Umstieg auf andere Systeme offen.

(jam@ct.de) **ct**

## Literatur

- [1] Jan Mahn, Zufall schlägt das System, Ein Plädoyer für UUIDs in Datenbanken, c't 6/2022, S. 138

**Dokumentationen:** [ct.de/y1n6](https://ct.de/y1n6)

**Suchspiel:** Fast alle in diesem Artikel vorgestellten Systeme haben eine Open-Source-Version. Auf den Websites muss man danach aber manchmal gründlich suchen. Wo kostenlos dransteht, ist oft nur ein Link aufs kostenfreie Cloudangebot drin. Die kostenlosen und lokal nutzbare Datenbanksoftware findet sich oft schneller auf GitHub.